

MRN Project 1: PHY/MAC Metrics Collection xApp

Giorgio Messina

giorgio.messina@mail.polimi.it

1 Overview

This report describes a Proof of Concept xApp for O-RAN that collects PHY and MAC layer metrics from a gNB emulator (gNBemu) through a custom, non 3GPP E2 Service Model carried over UDP with Protocol Buffers instead of ASN.1. The system has three parts: a protobuf schema shared by both endpoints, a Python xApp running against the near-RT RIC, and a C message handler on the gNB emulator side. Every 500 ms the xApp requests the RSRP, uplink and downlink BER, and uplink and downlink MCS of every connected UE, together with the current cell load (allocated PRBs against the maximum), and appends everything with a timestamp to two CSV files. I also wrote a small script to visualize the collected data, described in Section 5.

2 Protocol design

The schema follows the official Protocol Buffers style guide: CamelCase names for messages and enums (`RanMessage`, `UeInfo`, `UeList`, `RanParameterMapEntry`, ...), which maps predictably onto both the generated Python classes and, on the C side, the protobuf-c generated structs (`RanMessage` becomes `ran_message_pack`, `UeList` becomes `ue_list__init`, and so on).

```
1 enum RanParameterId {
2     GNB_ID = 1;
3     UE_LIST = 3;
4     GLOBAL_PRB_ALLOC = 5;
5     MAX_PRB = 6;
6 }
7 message UeInfo {
8     required uint32 rnti = 1;
9     optional float rsrp_dbm = 2;
10    optional float ber_ul = 3;
11    optional float ber_dl = 4;
12    optional uint32 mcs_ul = 5;
13    optional uint32 mcs_dl = 6;
14 }
```

`rnti`, `mcs_ul`, `mcs_dl` and `connected_ues` are `uint32`, since none of them can be negative. I left a few enum values and field numbers explicitly **reserved** for parameters I might add later (an extra cell or slice identifier, an additional measurement), so the numbering stays stable if the schema grows. `UeInfo` also carries two generic fields, `aux_flag` and `aux_value`, meant for control-oriented variants of this xApp (for example forcing a lower MCS or limiting the PRBs assigned to a UE) that are not exercised by this metrics-collection version.

3 gNB emulator

The emulator keeps a fixed-size fleet of simulated UEs and answers indication requests from the xApp with randomly generated but signal-consistent measurements. RSRP is sampled uniformly between -65 and -109 dBm (representative of an N78 deployment), and BER/MCS are then drawn from one of three bands depending on that RSRP value, since a UE with a strong signal should not realistically report a bad BER or a low MCS.

RSRP range	BER DL	BER UL	MCS range
≥ -80 dBm	0.000 to 0.004	DL + 0.001 to 0.005	20 to 27 (256-QAM)
-95 to -80 dBm	0.000 to 0.049	DL + 0.005 to 0.025	10 to 19 (64-QAM)
< -95 dBm	0.000 to 0.199	DL + 0.010 to 0.059 (cap 0.3)	0 to 9 (QPSK/16-QAM)

All the mutable state of the emulator (gNB id, PRB counters, initialization flag, the simulated UE array) is grouped into a single struct instead of being spread across file-scope globals, and the RSRP-to-BER/MCS logic above is split into three small functions (`classify_link_quality`, `sample_ber`, `sample_mcs`) instead of being inlined inside the function that builds the UE list:

```

1 static void sample_ber(link_quality_t quality, float *ber_ul, float *ber_dl) {
2     switch (quality) {
3         case LINK_QUALITY_STRONG:
4             *ber_dl = (rand() % 5) / 1000.0f;
5             *ber_ul = *ber_dl + 0.001f + (rand() % 5) / 1000.0f;
6             break;
7         case LINK_QUALITY_MEDIUM:
8             *ber_dl = (rand() % 50) / 1000.0f;
9             *ber_ul = *ber_dl + 0.005f + (rand() % 20) / 1000.0f;
10            break;
11        case LINK_QUALITY_WEAK:
12        default:
13            *ber_dl = (rand() % 200) / 1000.0f;
14            *ber_ul = *ber_dl + 0.01f + (rand() % 50) / 1000.0f;
15            break;
16    }
17    if (*ber_ul > BER_UL_CAP) *ber_ul = BER_UL_CAP;
18 }

```

Unrecognized or unimplemented message types are logged and dropped rather than triggering an `assert()`, since a component meant to run unattended should not abort just because it received something unexpected. The function that frees a UE list also sets every pointer to NULL right after freeing it, to avoid ever touching a freed pointer by mistake. Parameter ids are also mapped to a short name for the log lines, in the same spirit as the enum itself:

```

1 static const char *ran_parameter_name(RanParameterId param_id) {
2     switch (param_id) {
3         case RAN_PARAMETER_ID__GNB_ID: return "gnb_id";
4         case RAN_PARAMETER_ID__UE_LIST: return "ue_list";
5         case RAN_PARAMETER_ID__GLOBAL_PRB_ALLOC: return "global_prb_alloc";
6         case RAN_PARAMETER_ID__MAX_PRB: return "max_prb";
7         default: return "unrecognized_param";
8     }
9 }

```

Figure 1 shows the BER reported over a short test run against RSRP: the three sampling bands from the table above are clearly visible.

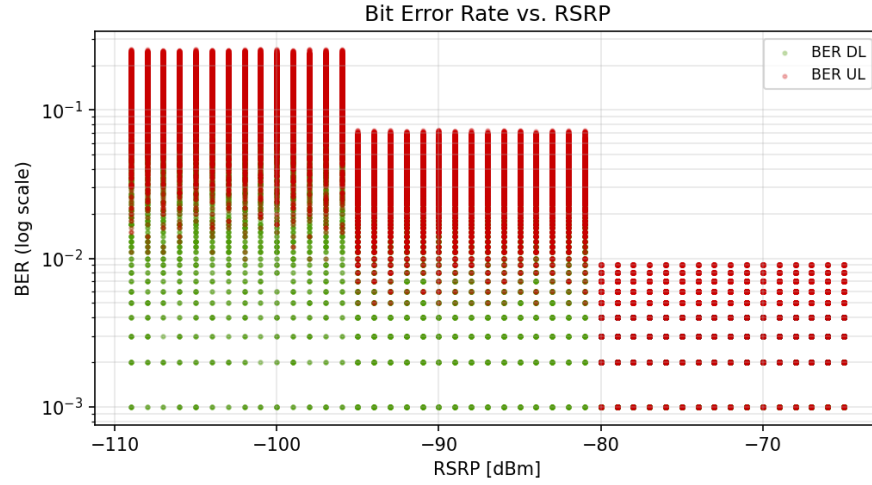


Figure 1: Downlink and uplink BER against RSRP. The three tiers used to sample BER and MCS appear as separate bands.

4 xApp

The xApp is organized into a few independent pieces instead of one large function: a small dataclass for the configuration (poll interval, output paths, requested parameters), a class that owns the two CSV files and knows how to append rows to them, a plain function that turns a decoded `RanIndicationResponse` into a small object with only the fields I need, and a class that ties discovery, subscription and the polling loop together. Keeping the parsing function independent of the RIC connection means it can be tested on its own, by building a fake response in memory, which is how I checked it behaves correctly without needing a live RIC:

```

1 def parse_indication_response(response):
2     snapshot = CellSnapshot()
3     for entry in response.param_map:
4         name = ransm.RanParameterId.Name(entry.key)
5         if name == "GNB_ID" and entry.HasField("string_value"):
6             snapshot.gnb_id = entry.string_value
7         elif name == "MAX_PRB":
8             snapshot.max_prb = entry.int64_value
9         elif name == "GLOBAL_PRB_ALLOC":
10            snapshot.allocated_prb = entry.int64_value
11        elif name == "UE_LIST":
12            snapshot.ue_list = list(entry.ue_list.ue_info)
13    return snapshot

```

Logging uses the standard `logging` module instead of `print`, and the poll interval and output folder are command line arguments (`--poll-interval`, `--out-dir`), defaulting to the 500 ms interval used throughout the project.

The two CSV files are:

```

1 e2sm_data.csv: timestamp, gnb_id, allocated_prb, max_prb, load
2 e2smue_data.csv: timestamp, gnb_id, rnti, rsrp_dbm, ber_ul, ber_dl, mcs_ul, mcs_dl

```

5 Data visualization

Since the xApp only writes CSV files, I wrote `visualize_ran_metrics.py` (pandas and matplotlib) to make sense of the collected data without having to open a spreadsheet. It prints a short summary (sample counts, number of distinct UEs, average load, average BER and MCS) and produces a few plots: cell load over time, RSRP distribution, BER against RSRP, MCS distribution, and a combined dashboard with all four. Figure 2 shows the cell load over a short run: allocated PRBs swing between nearly idle and fully loaded quite quickly, since each indication currently samples the load independently rather than following a smoother, more realistic traffic pattern.

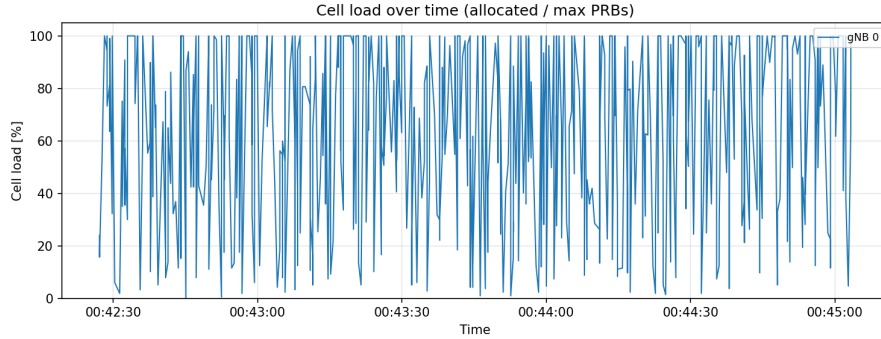


Figure 2: Cell load (allocated PRBs over the maximum) during a short test run.

6 Known limitations

The gNB handler was checked with `gcc -fsyntax-only` against a header written to match `protobuffer-c`'s naming conventions, not against a header actually generated by `protoc-c`, since that toolchain was not available while preparing this report. Before relying on this for a live demo, the natural next step is to regenerate `ran_messages.pb-c.h` with `protoc-c`, build the emulator against it, and run the full chain against a real near-RT RIC instance. The cell load and BER/MCS models are also independent random samples rather than derived from a shared SNR estimate, a more accurate version would compute BER from an estimated SNR and a link-adaptation table using the Q-function, which I left as a possible improvement.